

**Solution 1.** Below is the integer program requested.

$$\begin{array}{ll}
\text{Minimize} & y \\
\text{subject to} & c \in \{0, 1\}^{V \times V}, y \in \mathbb{Z}_+ \\
& \sum_{i=1}^n c_{iv} = 1, & \text{for each } v \in V. \\
& c_{iu} + c_{iv} \leq 1, & \text{for each } \{u, v\} \in E \text{ and } i \in V, \\
& y \geq \sum_{i=1}^n i c_{iv}, & \text{for each } v \in V.
\end{array}$$

The above formulation uses  $n^2$  variables where each one is associated with a vertex  $v$  and a color  $c$ , and 1 variable  $y$ , which is used to get the minimum-size coloring of  $G$ . If  $v$  has color  $c$ , the correspondent variable will be equal to 1, otherwise it will be 0. The first set of constraints — has  $n$  equalities and enforces that each vertex has one color. The second set of constraints — has  $n.m$  inequalities and guarantee that there is no neighboring vertices with the same color. Finally, the last set of constraints — has  $n$  inequalities and exists to extract the answer. For each inequality in this last set, exactly one variable will be equal to 1 and all the others equal to 0 (one vertex can have only one color). By that, the result of the sum will be  $i$ , where  $i$  is the index associated with the variable  $c_{i,v}$ , meaning that the vertex  $v$  has color  $i$ . Given that,  $y$  will be greater or equal to the maximum index  $j$  among all colors utilized. Since we are minimizing  $y$ , the best answer will get lower index colors, and thus  $y$  will be equal to the minimum number of colors possible to color  $G$ .

**Solution 2.** Suppose  $B_D$  is not totally unimodular. Then, there are  $X \subset V$  and  $Y \subset A$  such that  $|X| = |Y|$  and  $\det(B_D[X, Y]) \notin \{0, -1, +1\}$ . Among all other pairs  $(X, Y)$ , choose the one that minimizes the size of  $X$  and  $Y$ . If  $|X| = |Y| = 1$ ,  $\det(B_D[X, Y]) \in \{0, -1, +1\}$  because by definition all the entries of  $B_D$  are an element of  $\{0, -1, +1\}$ . Hence,  $|X| = |Y| \geq 2$ . If some column of  $B_D = 0$ , then  $\det(B_D[X, Y]) = 0$ . Hence, each column of  $B_D[X, Y]$  has at least one entry that is an element of  $\{-1, +1\}$ . If some column has exactly one 1 or one -1, say at entry  $(a, b) \in X \times Y$ , then, by Laplace expansion of the determinant, we have  $\det(B_D[X, Y]) = \pm \det(B_D[X \setminus \{a\}, Y \setminus \{b\}]) \in \{0, -1, +1\}$ . Hence, every column of  $B_D[X, Y]$  has exactly one entry equal to -1 and one entry equal to 1. Given that,  $(B_D[X \setminus \{a\}, Y \setminus \{b\}])^T \mathbb{1} = 0$ , where  $\mathbb{1}$  is the vector where all entries are equal to 1. Hence,  $(\det(B_D[X, Y]))^T = 0$ , and therefore,  $\det(B_D[X, Y]) = 0$ . Thus,  $B_D$  is totally unimodular.

---

**Solution 3.** First, it is impossible to have both objects at the same time. If you have a matching that saturates  $A$ , you have  $|A|$  different edges, one from each element in  $A$  to a different element in  $B$ . Therefore, if there is a matching that saturates  $A$ , the inequality  $|S| \leq |N(S)|$  holds for every  $S \subseteq A$  because every  $S \subseteq A$  has at least  $|S|$  elements in  $B$  that are neighbors of  $S$ . Also, if you have a subset  $S \subseteq A$  such that  $|S| > |N(S)|$  you can not have a matching that saturates  $A$  because there is a vertex  $v$  in  $S$  such that  $v$  is not an end of any edge on a maximum matching of  $G$ . This is easy to see because you can not have a matching that saturates  $S$  if you have fewer neighbors of  $S$  than  $|S|$ . If you can not have a matching that saturates  $S$ , you can not have a matching that saturates  $A$ .

The following algorithm solves the problem.

---

**Algorithm 1** Matching-or-Hall-Set ( $G, (A, B)$ )

---

```

1:  $(M, K) \leftarrow \text{Bip-Matching-and-Vertex-Cover}(G, (A, B))$ 
2: if  $|M| = |A|$  then
3:   return  $M$ 
4: end if
5:  $S := \emptyset$  ▷  $S$  will be the subset  $S \subseteq A$  such that  $|S| > |N(S)|$ 
6: for each  $v \in A$  do
7:   if  $v \notin K$  then ▷ this can be done in constant time by using an auxiliary array
8:      $S \leftarrow S \cup \{v\}$ 
9:   end if
10: end for
11: return  $S$ 

```

---

The call to Bip-Matching-and-Vertex-Cover algorithm with returns  $(M, K)$ , where  $M$  is a matching of  $G$  and  $K$  is a vertex cover of  $G$ , and  $|M| = |K|$ . By corollary 21, follow that  $M$  is a maximum matching of  $G$  and  $K$  is a minimum vertex cover of  $G$ .

Algorithm 1 runs Bip-Matching-and-Vertex-Cover algorithm, checks if  $M$  saturates  $A$ , and returns  $M$  if the verification is true. Otherwise, we know that no matching of  $G$  saturates  $A$ , and we need to return a subset  $S \subseteq A$  such that  $|S| > |N(S)|$ . In Algorithm 1, the subset  $S$  is defined as  $S := \{a \in A : a \notin K\}$ .

First,  $S$  is a subset of  $A$ . Second,  $N(S)$  is the set of all elements  $b \in B$  such that  $b \in K$ . If  $b \in B$  and  $b \in K$ , then  $b$  has at least one neighbor that is not in  $K$ , otherwise  $K$  would not be a minimum vertex cover because you could remove  $b$  from  $K$ , and it would still be a vertex cover, which is a contradiction. Therefore,  $b$  has a neighbor in  $S$  and, by definition, is in  $N(S)$ .

Finally, we will prove that  $|S| > |N(S)|$ . An important property of this setup is that  $|M| = |K|$  and  $|M| < |A|$ , thus  $|K| < |A|$ . Set  $K_A := \{a \in A : a \in K\}$  and  $K_B := \{b \in B : b \in K\}$ . Given that  $K_A$  and  $K_B$  are disjoint, we have  $|K_A| + |K_B| < |A|$ . Working on that, trying to have an inequality where one side is a subset of  $A$ , we have  $|K_B| < |A| - |K_A|$ . Because  $K_A \subseteq A$ , then  $|A| - |K_A| = |A \setminus K_A|$ . Knowing that  $K_B = N(S)$  and  $A \setminus K_A = S$ , it is proved that  $|N(S)| < |S|$ .

Regarding the time analysis, lines 2, 3, and 4 can be done in constant time if you store the size of the set (in the worst case, the IF verification is linear on the number of elements of the sets). The FOR loop on line 5 runs  $|A|$  times, and the IF on line 6 can be done in constant time by using an auxiliary boolean array that stores whether a vertex  $v$  is in  $K$  or not (it costs linear time on  $|V(G)|$  to build the array plus linear time on  $|K|$  to fill the array). Concluding, besides the running time of Bip-Matching-and-Vertex-Cover, Algorithm 1 runs in time  $O(n + m)$  and does not run any graph traversal algorithm.

**Solution 4.** Set  $C$  to be the set of all convex combinations of points of  $X$ . To prove that  $\text{conv}(X) = C$ , I will prove that  $\text{conv}(X) \subseteq C$  and that  $C \subseteq \text{conv}(X)$ .

To prove that  $\text{conv}(X) \subseteq C$ , I will prove that  $X \subseteq C$  and it is convex. Given that  $\text{conv}(X)$  is the intersection of all convex subsets of  $\mathbb{R}^V$  that contain  $X$ , if  $X \subseteq C$  and  $C$  is convex, then  $\text{conv}(X) \subseteq C$ .

First, a point  $y$  is convex combination of points of  $X_i$  if there is  $\lambda \in \mathbb{R}_+^I$  such that  $1\lambda = 1$  and  $y = \sum_{i \in I} \lambda_i x_i$ . Given this definition is easy to see that every  $x \in X$  is a convex combination of points of  $X$  (choosing  $\lambda_i = 1$  the sum will be equal to  $x_i$ ).

Finally, let  $y_1 := \sum_{i \in I} \alpha_i x_i$  and  $y_2 := \sum_{i \in I} \beta_i x_i$  be arbitrary convex combinations of points of  $X$  and let  $\lambda \in [0, 1]$ . We will prove that  $(1 - \lambda)y_1 + \lambda y_2$  is in  $C$ .

$$\begin{aligned} (1 - \lambda)y_1 + \lambda y_2 &= \left( \sum_{i \in I} (1 - \lambda) \alpha_i x_i \right) + \left( \sum_{i \in I} \lambda \beta_i x_i \right) \\ (1 - \lambda)y_1 + \lambda y_2 &= \sum_{i \in I} (1 - \lambda) \alpha_i x_i + \lambda \beta_i x_i \\ (1 - \lambda)y_1 + \lambda y_2 &= \sum_{i \in I} ((1 - \lambda) \alpha_i + \lambda \beta_i) x_i \end{aligned}$$

If  $\sum_{i \in I} ((1 - \lambda) \alpha_i + \lambda \beta_i) = 1$ , then  $(1 - \lambda)y_1 + \lambda y_2$  is a convex combination of points of  $X$ .

$$\begin{aligned} \sum_{i \in I} ((1 - \lambda) \alpha_i + \lambda \beta_i) &= \sum_{i \in I} (\alpha_i - \lambda \alpha_i + \lambda \beta_i) \\ \sum_{i \in I} ((1 - \lambda) \alpha_i + \lambda \beta_i) &= \sum_{i \in I} \alpha_i - \sum_{i \in I} \lambda \alpha_i + \sum_{i \in I} \lambda \beta_i \\ \sum_{i \in I} ((1 - \lambda) \alpha_i + \lambda \beta_i) &= 1 - \lambda + \lambda = 1 \end{aligned}$$

Thus, we proved that  $\text{conv}(X) \subseteq C$ .

To prove that  $C \subseteq \text{conv}(X)$ , I will prove that any convex combination of points of  $X$  is a point that is in every convex subset of  $\mathbb{R}^V$  that contain  $X$ , and therefore is in  $\text{conv}(X)$ .

By definition of convex,  $y_1 := \lambda_1 x_1 + (1 - \lambda_1)x_2$ , where  $x_1, x_2 \in X$  and  $\lambda_1 \in [0, 1]$ , is in  $\text{conv}(X)$ . Also,  $y_2 := \lambda_2 y_1 + (1 - \lambda_2)x_3$ , where  $x_3 \in X$  and  $\lambda_2 \in [0, 1]$ , is in  $\text{conv}(X)$  because  $y_1$  and  $x_3$  are points in  $\text{conv}(X)$ . Doing it recursively,

$$y := \left( \prod_{k \in I} \lambda_k \right) x_j + \left( \sum_{i \in I \setminus \{j\}} \left( \prod_{k \in I} \lambda_k \right) (1 - \lambda_i) x_i \right) + (1 - \lambda_i) x_l$$

is in  $\text{conv}(X)$  because of the definition of convexity.

First,

$$\left( \prod_{k \in I} \lambda_k \right) + \left( \sum_{i \in I \setminus \{j\}} \left( \prod_{k \in I} \lambda_k \right) (1 - \lambda_i) \right) + (1 - \lambda_i) = 1$$

Finally, any convex combination  $c := \sum_{i \in I} \lambda_i x_i$  can be written as ————. This follows because it is possible to build the coefficients on ———— based on  $\lambda_i$ . Choose an arbitrary  $x_1 \in X$  with coefficient  $\lambda_1 > 0$  and let  $(1 - \lambda_k) := \lambda_i$ . If  $\lambda_i = 1$ , then is trivial. If it  $\lambda_i < 1$ , then choose an arbitrary  $x_2 \in X$  with coefficient  $\lambda_2 > 0$

$$\sum_{j \in I \setminus \{i\}} \lambda_j > 0$$

**Solution 5.** To prove that

The edges of  $b(b(H))^\uparrow$  are all sets of vertices that have at least one vertex of each minimal vertex cover. Given that for a vertex cover you need at least one vertex that is an end of each hyperedge of the graph, you can use each vertex of the same hyperedge to get distinct minimal vertex covers. Also, it is important to state that every minimal vertex cover will have at least one of those vertices, otherwise it would not be a vertex cover. By that, every hyperedge of  $H$  is in  $b(b(H))^\uparrow$ . By the definition of the upperarrow symbol  $H^\uparrow \subset b(b(H))^\uparrow$ .

Edges of  $H^\uparrow$  are all sets of vertices that contain at least one hyperedge of  $H$ . Then, if a set of vertices is not in  $H^\uparrow$ , it means that it is a set that does not contain any hyperedge of  $H$ . Since there is a minimal vertex cover with each vertex of each hyperedge of  $H$ ,

---